

# LLM Agent 推理与规划综述：从 CoT 到 PDDL 的神经符号方法

五道口纳什 & Codex

2025-09-07

## Agent Planning

Reasoning models ...

Prompt-based: Plan-and-Solve, ...

Search-based: ToT, MCTS...

Test-time scaling: DeepConf

Neural-Symbolic: DSL/PDDL ...

视频作者/频道：五道口纳什

发布日期：2025-09-07

视频时长：32:11

视频链接：<https://www.bilibili.com/video/BV1wjY5zyEki>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

# 目录

|                                              |           |
|----------------------------------------------|-----------|
| <b>1 系列概述与课程规划</b>                           | <b>3</b>  |
| 1.1 本系列计划覆盖的主题                               | 3         |
| 1.2 本章小结                                     | 4         |
| <b>2 推理 (Reasoning): 模型智能的基石</b>             | <b>4</b>  |
| 2.1 推理的三个经典例子                                | 4         |
| 2.2 推理的两种基本形式                                | 5         |
| 2.2.1 演绎推理 (Deduction)                       | 5         |
| 2.2.2 归纳推理 (Induction)                       | 5         |
| 2.3 隐式推理与长链思考                                | 5         |
| 2.4 本章小结                                     | 6         |
| <b>3 规划 (Planning): 任务分解与长期求解</b>            | <b>6</b>  |
| 3.1 什么是规划                                    | 6         |
| 3.2 2024 年 LLM Planning Survey               | 6         |
| 3.2.1 分解 (Decomposition)                     | 7         |
| 3.2.2 选择 (Selection)                         | 7         |
| 3.2.3 外部求解器 (External Solver)                | 7         |
| 3.2.4 反思 (Reflection / Critic)               | 7         |
| 3.3 本章小结                                     | 7         |
| <b>4 经典 Workflows 与架构模式</b>                  | <b>7</b>  |
| 4.1 ReAct: 推理-行动循环                           | 7         |
| 4.2 Plan-and-Solve (PS) 与动态重规划               | 8         |
| 4.2.1 Zero-Shot CoT → PS 的演进                 | 8         |
| 4.2.2 缸中之脑与真实交互的矛盾                           | 9         |
| 4.3 Tree of Thoughts (ToT) 与 MCTS            | 9         |
| 4.4 本章小结                                     | 10        |
| <b>5 神经符号方法 (Neural-Symbolic): LM + PDDL</b> | <b>10</b> |
| 5.1 为什么需要 PDDL?                              | 10        |
| 5.2 PDDL 的典型工作流                              | 10        |
| 5.3 Block World 问题的典型困境                      | 11        |
| 5.4 一个具体例子: 放置笔记本                            | 11        |
| 5.5 AlphaGeometry: 神经符号的经典案例                 | 11        |
| 5.6 本章小结                                     | 12        |
| <b>6 测试时计算 (Test-Time Compute)</b>           | <b>12</b> |
| 6.1 DeepConf: 置信度加权的多数投票                     | 12        |
| 6.2 UCLA IMO 解决方案: Self-Critic Workflow      | 13        |
| 6.3 本章小结                                     | 13        |
| <b>7 Memory 中的反思 (Reflection)</b>            | <b>13</b> |

|                         |           |
|-------------------------|-----------|
| <b>8 总结与延伸</b>          | <b>13</b> |
| 8.1 主讲人核心观点总结 . . . . . | 13        |
| 8.2 一个发人深省的例子 . . . . . | 14        |
| 8.3 实用建议 . . . . .      | 14        |
| 8.4 系列预告 . . . . .      | 14        |

# 1 系列概述与课程规划

本视频是【Modern Agent】系列的第一期，定位为对整个系列的宏观预览。主讲人五道口纳什计划从最基础的底座模型能力出发（推理与规划），逐步深入到经典的 Agentic Workflow、Context Engineering、Memory 设计、Self-Evolving 技术、框架使用（LangChain/LangGraph）、知识图谱方法（GraphRAG）、神经符号系统（Neural-Symbolic）、工具封装等主题。

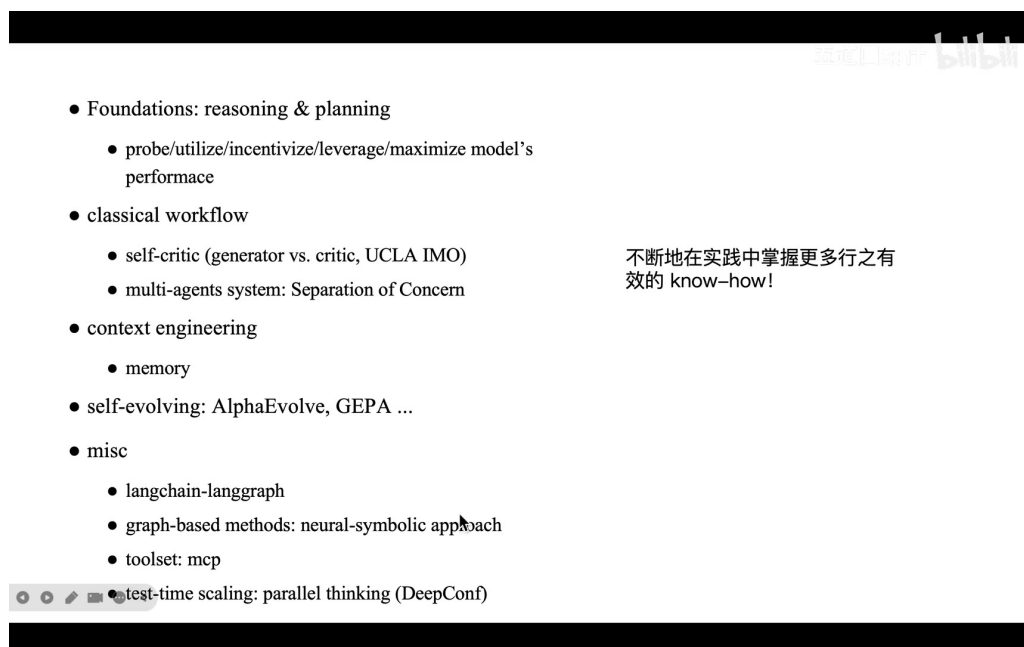


图 1: 本系列覆盖的主要话题总览<sup>1</sup>

## 1.1 本系列计划覆盖的主题

1. **底座模型能力**：推理（Reasoning）与规划（Planning）——这是后续所有内容的基石。
2. **经典 Workflow**：Self-Critic、Multi-Agent System（关注 Separation of Concern / 任务分解中的责任分离）。
3. **Context Engineering**：尤其是 Memory 设计，以及 Self-Evolving 技术（如 Tool Evolve 和 JJP）。
4. **框架与工具**：LangChain / LangGraph 的使用，Graph-based 方法（GraphRAG），Neural-Symbolic 方案。
5. **工具封装**：为语言模型设计合适的 Tool 或 Server，弥补 LMs 不擅长的领域。
6. **Test-Time Compute**：并行思考（以 DeepSeek R1 为代表）、多数投票（Majority Voting）等技术。

<sup>1</sup> 视频画面时间区间：00:02:50–00:03:05

### 核心观点

模型真正的智能通过推理（Reasoning）来体现。Agentic 是对模型智能的兑现——通过频繁地与外部环境交互去求解问题，让模型的推理能力真正落地。

## 1.2 本章小结

本系列致力于在实践中掌握更多行之有效的 Know-How。第一期作为串讲，为大家建立对整个领域的宏观认知框架。

## 2 推理（Reasoning）：模型智能的基石

### 什么是推理（Reasoning）

推理是一种无中生有的能力——从已知且有限的前提（Premise）出发，沿着推理链推导出更多、更丰富、更有深度的结论。它是语言模型各项能力的核心基础。

### 2.1 推理的三个经典例子

- **演绎推理三段论**：大前提（所有人都会死）→ 小前提（苏格拉底是人）→ 结论（苏格拉底也会死）。
- **系统提示词中的推理**：在 Paper Reading 场景中，设计 System Prompt 让模型理解论文 → 生成评论 → 整体锐评 → ”One More Thing” 自由发挥，极度依赖模型的推理能力从已有内容中推理出非浅层的新结论。
- **Reasoning 与规划的关系**：如果模型没有推理能力，就没有规划（Planning）。因为规划涉及任务分解、提前思考（Think Ahead）、自我反思（Reflection）和批评（Critic），这些都需要大量的推理生成结论。

## 2.2 推理的两种基本形式

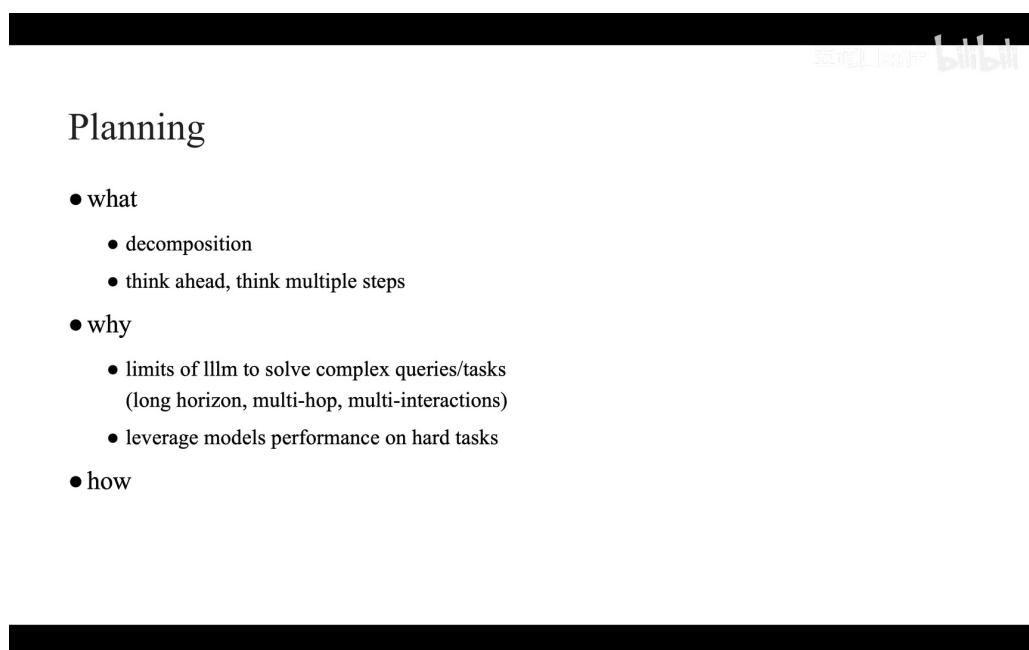


图 2: Induction（归纳推理）与 Deduction（演绎推理）的对比<sup>2</sup>

### 2.2.1 演绎推理 (Deduction)

- 前向推理，基于三段论形式。
- 确定性推理——前提为真则结论必然为真。
- 示例：大前提 + 小前提 → 结论。

### 2.2.2 归纳推理 (Induction)

- 或然性推理——前提越多，结论越自信，但非绝对确定。
- 示例：见过的天鹅都是白色的 → 结论” 天鹅都是白色的”（或然性）。

#### 关键区分

演绎是**确定性**推理，归纳是**或然性**推理。在 Agent 设计中，需要明确识别任务所需的推理类型，避免将归纳结论当作确定性事实使用。

## 2.3 隐式推理与长链思考

以 DeepSeek R1 和 OpenAI OO 系列为代表的推理模型，追求一种**隐式**的 Long CoT (Chain-of-Thought)：

- 复杂问题 → 模型内部自发产生长的思考过程。
- 简单问题 → 产生短的思考过程。
- 这是一种隐式的 CoT，模型在训练时被训练为自发地产生不同长度的推理链。

<sup>2</sup>视频画面时间区间：00:08:30–00:08:45

## 2.4 本章小结

Reasoning 是 Planning 的基础。没有推理能力，模型也无法完成规划、反思或自我批评。R1/OO 系列代表了隐式推理模型的最新进展，而显式的 Prompt Engineering 方法（如 CoT、ReAct）则提供了更可控的推理路径。

## 3 规划 (Planning)：任务分解与长期求解

### 3.1 什么是规划

#### 定义

规划 (Planning) 是对任务的分解——无论是显式还是隐式，都需要：识别用户目标 → 分解为子步骤 → 确定执行顺序 → 处理约束条件。

规划涉及：

- **Task Decomposition**：将复杂 Query 拆解为可执行的子任务。
- **Single Head / Multi-Step Thinking**：提前想很多步。
- **长周期 Multi-Hop 问题**：多轮交互中，语言模型存在缺陷和不足，可以通过显式或隐式的规划来最大化模型性能。

### 3.2 2024 年 LLM Planning Survey

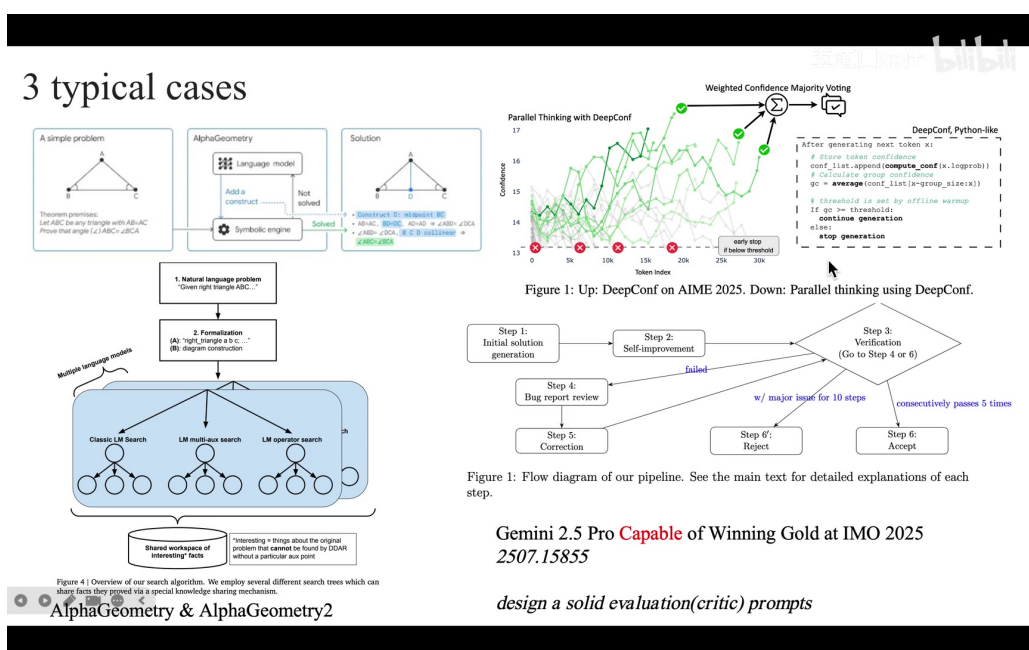


图 3: LLM Planning Survey 的四大模块分类<sup>3</sup>

2024 年初有一篇非常经典的 Survey，将 LLM Planning 分为以下几个核心模块：

<sup>3</sup>视频画面时间区间：00:14:30–00:15:00

### 3.2.1 分解 (Decomposition)

- **CoT (Chain-of-Thought)**: Let's think step by step——逐步求解而非直接给答案，内部涉及对问题的自由分解。
- **ReAct**: 思考 → 调用工具 → 得到响应 → 再思考 → 再循环，在交互过程中完成任务分解。
- **Plan-and-Solve (PS)**: 显式地形成一个 Plan，然后沿着 Plan 去求解问题。

### 3.2.2 选择 (Selection)

- **ToT (Tree of Thoughts)**: 用树管理搜索过程，从根节点出发做叶子节点的扩展。
- **MCTS (Monte Carlo Tree Search)**: 基于 UCB 值选择最有潜力的节点进行探索。
- 核心挑战: 状态爆炸 (指数级)，需要类似 Beam Search 的剪枝策略。

### 3.2.3 外部求解器 (External Solver)

- 语言模型直接产生 Plan 时往往不可靠。
- **LLM + P (PDDL Planner)**: 让语言模型做翻译和状态定义，PDDL Solver 做精确规划。
- 这是 Neural-Symbolic 方案的一个典型代表。

### 3.2.4 反思 (Reflection / Critic)

- 有了反思就可以做 Self-Critic: 提 Proposal → Critic 验证 → 反馈 → 再生成循环。
- Memory 中也可以引入反思——不仅记住事实，还要在事实基础上做更高层次的推理。

## 3.3 本章小结

LLM Planning 可以按 Decomposition → Selection → External Solver → Reflection 四个维度去理解和分类。每种方法都有其适用场景和局限。

## 4 经典 Workflows 与架构模式

### 4.1 ReAct: 推理-行动循环

ReAct 是最基础的 Agent 模式之一:

1. 基于当前 Reasoning/Thought。
2. 调用 Tool 或执行 Action。
3. 拿到 Tool 的 Response / 新的观测。
4. 循环上述过程直到问题解决。

## ReAct 的局限性

当交互过程特别长时，会出现 Context Issues：

- 模型会 Lost in the Middle——迷失在上下文中。
- 忘记用户的原始任务是什么。
- 忘记自己做过哪些事情、得到了什么结果。
- 不足以产生下一步合理的 Action/Plan。

可以通过现代 Context Engineering 技巧来缓解，但无法完全消除。

## 4.2 Plan-and-Solve (PS) 与动态重规划

### 4.2.1 Zero-Shot CoT → PS 的演进

Plan-and-Solve 是对 Zero-Shot CoT 的改进：

1. 先生成一个 Plan（显示地在 Prompt 中引入规划概念）。
2. 严格按照 Plan 执行。
3. 可引入动态性（Dynamic Plan-and-Solve）。

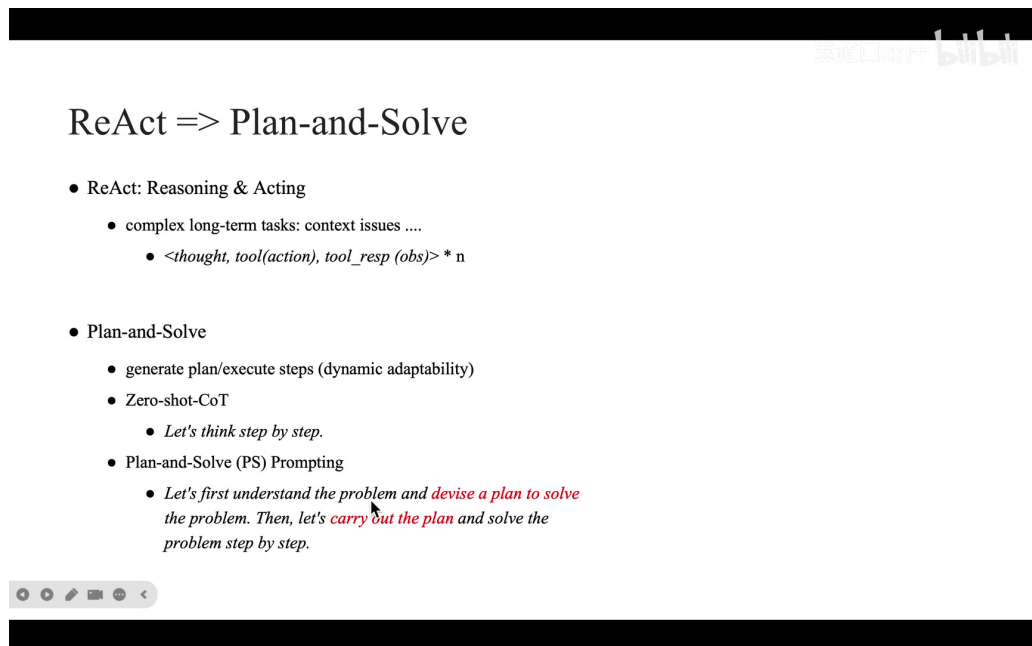


图 4: ReAct → Plan → Execute → RePlan 的 Workflow 架构<sup>4</sup>

<sup>4</sup>视频画面时间区间：00:22:30–00:22:50

## 4.2.2 缸中之脑与真实交互的矛盾

### 核心矛盾

Plan 是在头脑中形成的，没有和环境真实交互。当与真实环境交互时，会有新的 Feedback。这时需要把这个 Feedback 引入 Plan 机制，进行动态的 Revise 或 Replan。

动态 Plan-and-Solve 的核心流程：

1. **Planner** 形成 Plan (List of Steps)。
2. 每个 Step 交给 **ReAct** 去执行。
3. ReAct 执行完后，检查是否解决了问题。
4. 若未解决，**RePlan** 产生新的 Steps。

**经济性考虑：**用强的（贵的）模型做 Planner 和 RePlan，用弱的（便宜的）模型做具体 ReAct 的执行。这是从经济角度组合模型的有效策略。

## 4.3 Tree of Thoughts (ToT) 与 MCTS

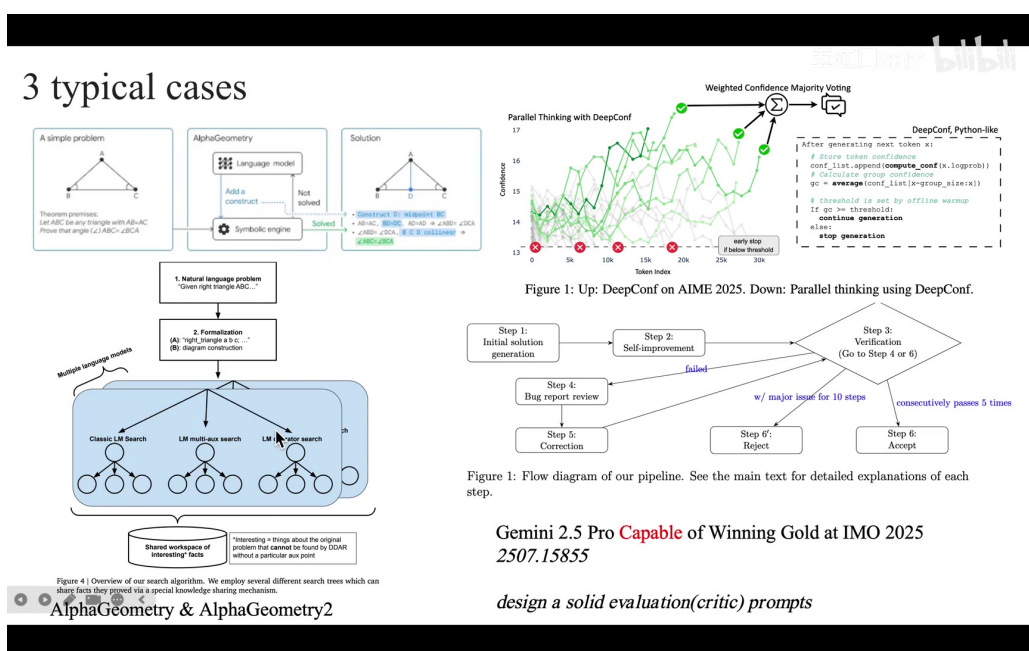


图 5: ToT 与 MCTS：用树结构管理语言模型的搜索过程<sup>5</sup>

ToT 和 LATS (LM + MCTS) 的核心在于 **Selection**：

1. 从一个节点出发做扩展，语言模型在该节点产生多个 Proposal。
2. 状态爆炸问题——需要对节点打分并过滤。
3. 基于 UCB 值（探索 vs 利用的平衡）选择最有潜力的节点继续搜索。
4. Score/Critic 机制的设计非常关键，直接影响搜索质量。

<sup>5</sup> 视频画面时间区间：00:17:30–00:17:50

## 4.4 本章小结

从 ReAct → Plan-and-Solve → ToT/MCTS, Agent 架构从简单的交互循环发展到显式规划再到树搜索, 复杂度逐步提升, 但每种模式都有自己的适用边界。

## 5 神经符号方法 (Neural-Symbolic): LM + PDDL

### 5.1 为什么需要 PDDL ?

#### 核心洞察

语言模型擅长做翻译 (Translation) 和判定 (Judgment), 但不擅长直接产生精确的 Plan。PDDL 有稳健的 Solver, 可以产生真实可行的执行序列。将两者结合 (LM + P) 就是扬长避短。

### 5.2 PDDL 的典型 workflow

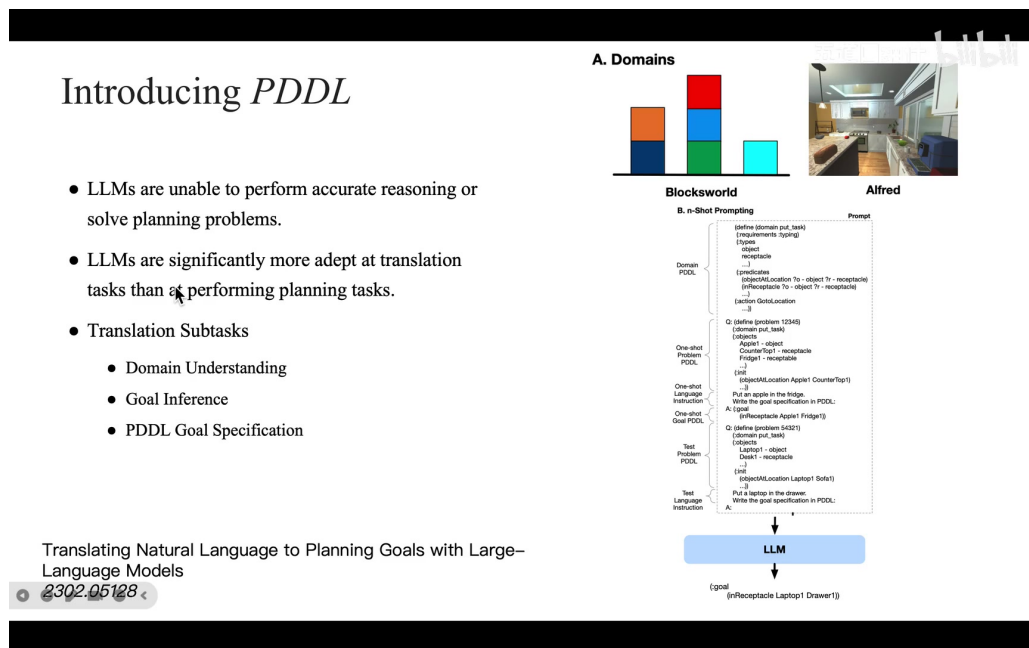


图 6: LM + P (PDDL Planner) 的 Neural-Symbolic workflow<sup>6</sup>

以机器人物体操作为例:

- 问题定义:** 用户描述一个操作任务 (如” 把苹果放在冰箱里” )。
- LM 做翻译:** 语言模型将自然语言描述翻译为 PDDL Goal State。
- PDDL Solver 求解:** 基于 Domain PDDL 定义和 Goal State, 求解器生成精确的 Action Sequence。
- LM 做逆向翻译:** 将 PDDL 执行序列翻译回自然语言, 呈现给用户。

<sup>6</sup>视频画面时间区间: 00:27:30–00:28:00

## LM as Planner vs LM + P

- **LM as Planner (直接产生 Plan)**: 效果较差, 容易产生步骤缺失或多余步骤。
- **LM + In-Context Examples**: 提供示例 Problem 和 Solution 后略好, 但仍不可靠。
- **LM + P (LM 只产生 Goal State, PDDL Solver 求解 Plan)**: 效果最好, 充分利用了语言模型的翻译能力和 PDDL Solver 的精确规划能力。

### 5.3 Block World 问题的典型困境

原始问题示例: 五在三上面, 三在四上面, 四在二上面, 二在一上面——要用机械臂达到新的目标状态。GPT-4 直接产生 Plan 时, 第四步就出错了。因为涉及状态约束 (机械臂一次只能拿一个 Block, Block 必须在桌面或特定位置), 语言模型很容易在内部产生有明显错误的 Plan。

### 5.4 一个具体例子: 放置笔记本

用户意图: ”把笔记本放在抽屉里”。

1. LM 产生 Goal State: 笔记本 1 在抽屉 1 里面。
2. Goal State 用 PDDL 语法表示。
3. PDDL Solver 基于 Domain 定义和 Goal State 进行精确求解。
4. 输出精确的 Action Sequence (抓取 → 移动 → 放置)。

### 5.5 AlphaGeometry: 神经符号的经典案例

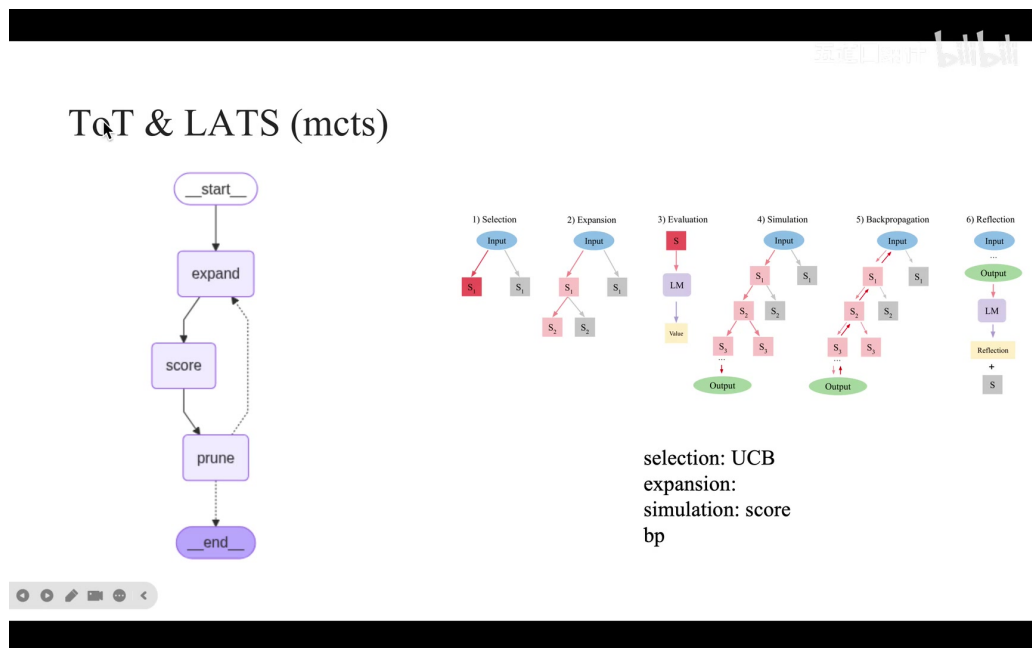


图 7: AlphaGeometry 第一代与第二代架构对比<sup>7</sup>

<sup>7</sup> 视频画面时间区间: 00:25:30-00:26:00

### AlphaGeometry 第一代:

1. 将几何问题翻译为符号语言。
2. 符号引擎尝试直接求解。
3. 若不能求解, LM 添加辅助线 (如取 BC 中点 D, 连接 AD)。
4. 基于三角形相似等定理, 符号引擎完成证明。

### AlphaGeometry 第二代:

1. 引入多个并行的 LM 提 Proposal 和搜索思路。
2. 多路并行思考: 经典模型搜索 + 多路辅助线搜索 + ...
3. 关键创新: 中间过程中符号引擎推理出的新 Facts 是共享的, 可以成为其他模型的新前提 (States), 形成多路并行搜索的协作。

### 对 DeepSeek 的启发

虽然 DeepSeek 在自然语言层面 (无需 Symbolic Solver) 就实现了 IMO 金牌水平, 但 AlphaGeometry 2 的并行思考策略启发了他们如何设计更好的 Parallel Thinking 策略。

## 5.6 本章小结

Neural-Symbolic 方案 (LM + P) 通过**扬长避短**——让 LM 做擅长的事 (翻译、判定), 让精确求解器做擅长的事 (生成可靠 Plan) ——是目前 Agent Planning 领域最实用的方法之一。

## 6 测试时计算 (Test-Time Compute)

### 6.1 DeepConf: 置信度加权的多数投票

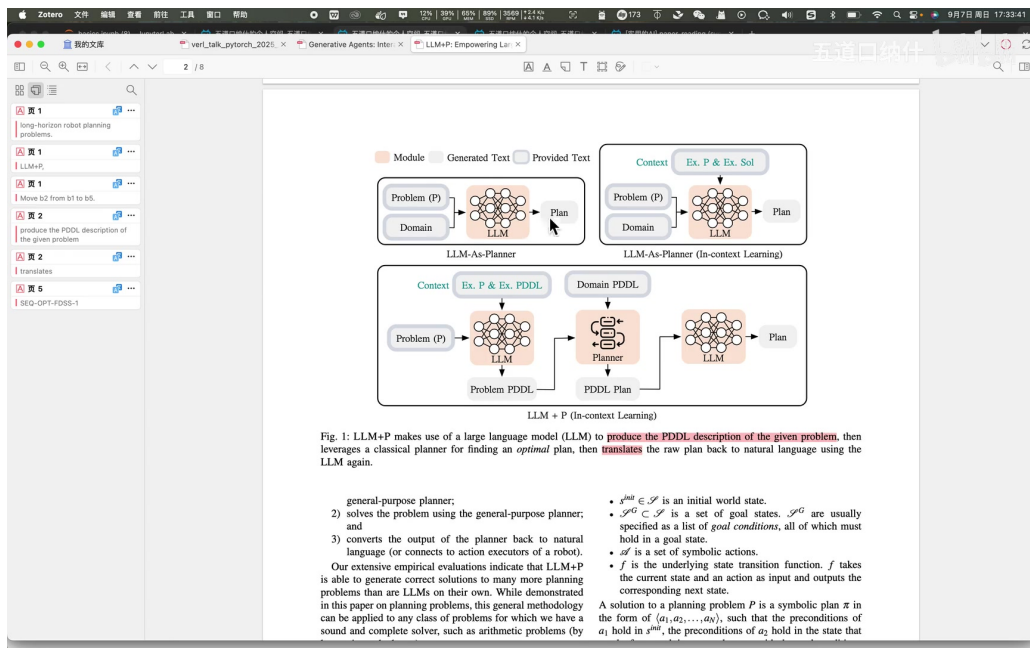


图 8: DeepConf: 对每个 Trace 计算置信度, 做加权多数投票<sup>8</sup>

## 6.2 UCLA IMO 解决方案: Self-Critic Workflow

### Self-Critic 核心流程

1. **Step 1-2**: Generator 提一个 Proposal (方案)。
2. **Step 3**: Critic 做 Verification——给出具体问题和意见。
3. **Step 4**: Generator 基于 Critic 反馈 Refine 方案。
4. **循环**: 直到连续 5 次评估结果为”完美” → 接受该解。
5. 若连续 10 次有重大问题 → 拒绝该方案。

## 6.3 本章小结

Test-Time Compute 的核心思想是在推理阶段投入更多计算资源来提高输出质量。Majority Voting、DeepConf、Self-Critic 都是这一方向的代表方法。

## 7 Memory 中的反思 (Reflection)

以 Generative Agents 为例, 在维护 Memory 时, 仅记住事实是浅层的、无价值的。需要在 Memory 层面做一次 Reflection:

你上午遇到一个人在读论文, 下午发现他也在读论文, 晚上再遇到时他还在读论文。此时触发一次反思, 语言模型得出更高层次的结论: **这是一个对科研非常有热情的人。**

### Memory 中 Reflection 的价值

仅记住事实 → 浅层记忆, 无助深入思考。基于多个事实触发 Reflection → 形成更有深度的更高层次的结论 → 才能真正帮助 Agent 进行有深度的规划。

## 8 总结与延伸

### 8.1 主讲人核心观点总结

1. **显式或隐式的 Plan 非常必要且有效**, 对于语言模型求解复杂的长周期问题至关重要。
2. 涉及 **Top-Down** 和 **Bottom-Up** 两个过程:
  - Top-Down: 任务分解形成 Plan。
  - Bottom-Up: Revise 或 RePlan, 弥补 Plan 和执行之间的 Gap (缸中之脑与真实交互的矛盾)。
3. **扬长避短**: 感知模型的能力及特点, 设计合适的 Workflow, 而不是期待语言模型解决所有问题。
  - 语言模型擅长: 翻译、判定、生成多样化的 Proposal。

<sup>8</sup>视频画面时间区间: 00:29:30-00:29:50

- 语言模型不擅长：精确推理、多步状态约束、长周期精确执行。

4. **系统级设计**：语言模型、Tools、Prompts 需要作为一个系统来设计。

## 8.2 一个发人深省的例子

### 即使最强模型也会犯错

即使强如 Gemini 2.5 Pro，解方程“ $5.9 = x + 5.11$ ”也会犯错。我们不当预期语言模型能解决所有问题。解决方案很简单——让一个计算器去做精确计算，语言模型只需要知道什么时候应该调用计算器。

## 8.3 实用建议

- 在 Agent 设计中，要**感知模型的边界**，不盲目信任模型的输出。
- 设计 Workflow 时要**预留人工验证或外部验证的环节**。
- 对于状态空间明确的任务，优先考虑 **Neural-Symbolic 方案** (LM + PDDL / 外部 Solver)。
- **经济性**：组合强模型（规划/决策）和弱模型（执行）以降低成本。
- **从已有工作中学习**：UCLA IMO 方案、AlphaGeometry、DeepConf 等工作提供了丰富的设计范式。

## 8.4 系列预告

后续视频将深入讲解：

- LangChain / LangGraph 在实现 ReAct 中的差异
- Plan-Execute-RePlan 的具体实现
- Context Engineering 与 Memory 设计
- Claude Code Memory 系统
- 更多 Neural-Symbolic 实践案例